

1. Can you have multiple contexts in a single React application?

Yes, you can have multiple contexts in a React app. Each context can store different types of global data — for example:

- ThemeContext → for dark/light mode
- AuthContext → for login info
- LanguageContext → for selected language

How to consume multiple contexts in a component

You can use multiple useContext hooks in the same component:

```
import {{ useContext }} from "react";

import {{ ThemeContext }} from "../ThemeContext";
import {{ AuthContext }} from "../AuthContext";

function Dashboard() {{
  const {{ theme }} =
    useContext(ThemeContext);
  const {{ user }} =
    useContext(AuthContext);

  return (
    <div className={{theme === "dark" ? "dark-mode" :
"light-mode"}}>
      <h2>Welcome, {{user.name}}</h2>
    </div>
  );
}}
```

You can also nest multiple Providers:

```
<AuthProvider>
  <ThemeProvider>
    <App />
  </ThemeProvider>
</AuthProvider>
```

2. Best practices when using React Context and Reducer

When using Context

- Keep context small – use it only for truly global data.
- Avoid re-render problems – split contexts if they become too large.
- Combine with custom hooks – e.g. useAuth = () => useContext(AuthContext).
- Don't use Context for every state – prefer useState for local ones.

When using Reducer

- Define clear action types (use constants).
- Keep reducer pure – no side effects inside the reducer.
- Use switch or mapping for readability.
- Combine Context + Reducer for global state management.
- Keep initial state well structured.

This approach makes React apps scalable and maintainable.